

23AID303 - Quantum Computing and Algorithms

Lab 3

1. Implementation of Full-Adder using Quantum Gates

Basic code

```
%matplotlib inline

from qiskit import *

from qiskit.visualization import plot_bloch_multivector
from qiskit.visualization import plot_histogram
from qiskit_aer import Aer
from qiskit.quantum_info import Statevector
import matplotlib.pyplot as plt

# Creating a circuit with 8 quantum bits and 2 classical bits
qc = QuantumCircuit(5,2)

# Preparing inputs
qc.x(0) # Comment this line to make Qbit0 = |0>
qc.x(1) # Comment this line to make Qbit1 = |0>
qc.x(2) # Comment this line to make Qbit2 = |0> ( carry-in bit )
qc.barrier() #The barrier function creates a visual separation making the
              #circuit diagram more readable, and it also prevents Qiskit from
              #performing various simplifications and optimizations across barriers during
              #compilation when circuits are run on real hardware

# AND gate1 implementation
```

```
# Sum calculation
```

```
qc.cx(0, 3)    #  $q_3 = A$ 
```

```
qc.cx(1, 3)    #  $q_3 = A \oplus B$ 
```

```
qc.cx(2, 3)    #  $q_3 = A \oplus B \oplus C_{in}$ 
```

```
qc.barrier()
```

```
# Carry calculation
```

```
qc.ccx(0, 1, 4) # Carry from A & B
```

```
qc.cx(0, 1)    # temporary:  $A \oplus B$  in  $q_1$ 
```

```
qc.ccx(1, 2, 4) # Carry from  $(A \oplus B)$  &  $C_{in}$ 
```

```
qc.cx(0, 1)    # restore original B
```

```
# ===== GET STATEVECTOR AND PLOT BLOCH SPHERE =====
```

```
# Instead of running simulator manually, use Statevector.from_instruction
```

```
state = Statevector.from_instruction(qc)
```

```
plot_bloch_multivector(state)
```

```
plt.show()
```

```
# Measure outputs
```

```
qc.measure(3, 0) # Sum  $\rightarrow$  classical bit 0
```

```
qc.measure(4, 1) # Carry  $\rightarrow$  classical bit 1
```

```
qc.barrier()
```

```
# Draw circuit
```

```
qc.draw(output='mpl')
```

```
# Run simulation
```

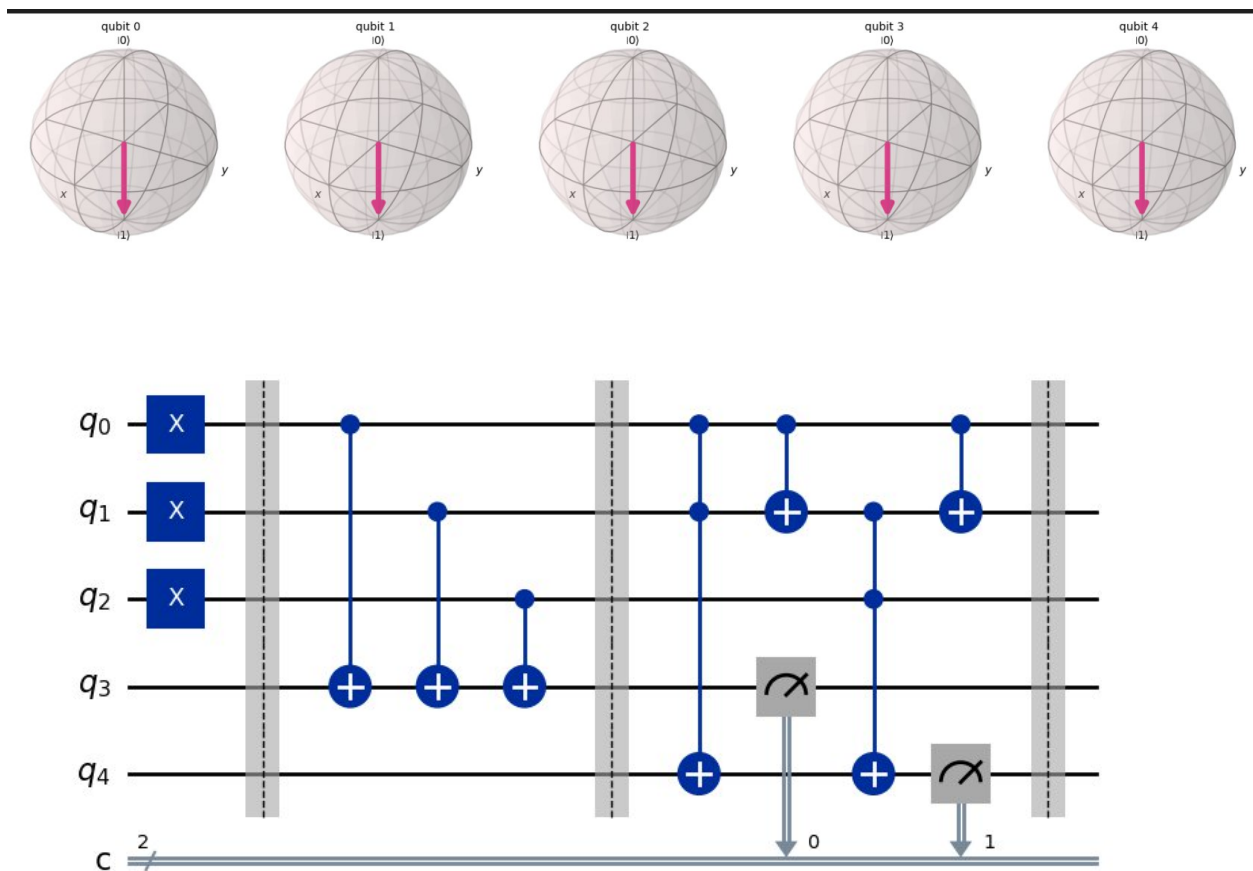
```

simulator = Aer.get_backend('qasm_simulator')
cc=transpile(qc, simulator)
# Run the experiment 1024 times and get stats
counts = simulator.run(cc,shots=1024).result().get_counts()

# Plot histogram
plot_histogram(counts)
plt.show()

print(counts)

```



2. Full Adder to print all combinations

```
from qiskit import *
from qiskit.visualization import plot_bloch_multivector, plot_histogram
from qiskit_aer import Aer
from qiskit.quantum_info import Statevector
import matplotlib.pyplot as plt

# Create circuit: 5 qubits, 2 classical bits (sum and carry)
qc = QuantumCircuit(5, 2)

# Put input qubits q0, q1, q2 in superposition to generate all input combos
qc.h(0)
qc.h(1)
qc.h(2)
qc.barrier()

# Full adder sum calculation (q3 = sum)
qc.cx(0, 3)    # q3 = A
qc.cx(1, 3)    # q3 = A  $\oplus$  B
qc.cx(2, 3)    # q3 = A  $\oplus$  B  $\oplus$  Cin
qc.barrier()

# Carry calculation (q4 = carry)
qc.ccx(0, 1, 4) # carry from A & B
qc.cx(0, 1)    # temporary: A  $\oplus$  B in q1
qc.ccx(1, 2, 4) # carry from (A $\oplus$ B) & Cin
```

```

qc.cx(0, 1)    # restore original B
qc.barrier()

# Measure sum and carry to classical bits
qc.measure(3, 0) # sum
qc.measure(4, 1) # carry

# Draw the circuit
qc.draw(output='mpl')

# Run simulation
simulator = Aer.get_backend('qasm_simulator')
compiled_circuit = transpile(qc, simulator)
result = simulator.run(compiled_circuit, shots=1024).result()
counts = result.get_counts()

# Plot histogram of output states
plot_histogram(counts)
plt.show()

print(counts)

```

3. Deutsch Algorithm

```

from qiskit import *
from qiskit_aer import Aer
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt

```

```

# Define oracle for a given f(x)
# f_type: "constant0", "constant1", "balanced01", "balanced10"
def deutsch_oracle(f_type):
    qc = QuantumCircuit(2)
    if f_type == "constant1":
        #  $f(0) = 1, f(1) = 1 \rightarrow$  flip output qubit always
        qc.x(1)
    elif f_type == "balanced01":
        #  $f(0) = 0, f(1) = 1 \rightarrow$  CNOT from x to y
        qc.cx(0, 1)
    elif f_type == "balanced10":
        #  $f(0) = 1, f(1) = 0 \rightarrow$  flip y, then CNOT
        qc.x(1)
        qc.cx(0, 1)
    # constant0 does nothing
    return qc

```

```

# Deutsch algorithm circuit
def deutsch_algorithm(f_type):
    qc = QuantumCircuit(2, 1)
    # Step 0: Initialize  $|0\rangle|1\rangle$ 
    qc.x(1) # make second qubit  $|1\rangle$ 
    # Step 1: Apply Hadamards
    qc.h([0, 1])
    #qc.barrier()
    # Step 2: Apply Oracle
    qc.compose(deutsch_oracle(f_type), inplace=True)

```

#compose() doesn't have a symbol in the diagram — it simply inserts whatever gates are in the subcircuit at the point of call it

#If the subcircuit has one gate, we see one gate. If it has three gates, we see three gates.

Step 3: Hadamard on first qubit

```
qc.h(0)
```

Step 4: Measure first qubit

```
qc.measure(0, 0)
```

```
return qc
```

Test with a balanced/constant function

```
qc = deutsch_algorithm("constant1")
```

```
qc.draw('mpl')
```

Run on simulator

```
sim = Aer.get_backend('qasm_simulator')
```

```
job = sim.run(qc, shots=1024)
```

```
result = job.result()
```

Get the counts

```
counts = result.get_counts(qc)
```

```
print("Measurement counts:", counts)
```

Decide constant or balanced

```
if '0' in counts and counts['0'] > counts.get('1', 0):
```

```
    print("Detected: CONSTANT function")
```

```
else:
```

```
    print("Detected: BALANCED function")
```

```
# Plot histogram
plot_histogram(counts)
plt.show()
```

4. Version 2 of Deutsch Algorithm (to check for all categories)

```
%matplotlib inline
from qiskit import *
from qiskit_aer import Aer
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt
```

```
# Oracle
def deutsch_oracle(f_type):
    qc = QuantumCircuit(2)
    if f_type == "constant1":
        qc.x(1)
    elif f_type == "balanced01":
        qc.cx(0, 1)
    elif f_type == "balanced10":
        qc.x(1)
        qc.cx(0, 1)
    # constant0 → do nothing
    return qc
```

```
# Deutsch algorithm
def deutsch_algorithm(f_type):
```

```

qc = QuantumCircuit(2, 1)
qc.x(1)          # Step 0: Initialize  $|0\rangle|1\rangle$ 
qc.h([0, 1])     # Step 1: Hadamards
qc.compose(deutsch_oracle(f_type), inplace=True) # Step 2: Oracle
qc.h(0)          # Step 3: Hadamard on first qubit
qc.measure(0, 0) # Step 4: Measure
return qc

```

```

# All possible  $f(x)$  types

```

```

f_types = ["constant0", "constant1", "balanced01", "balanced10"]

```

```

results = {}

```

```

sim = Aer.get_backend('qasm_simulator')

```

```

for f in f_types:

```

```

    qc = deutsch_algorithm(f)
    job = sim.run(qc, shots=1024)
    counts = job.result().get_counts(qc)
    results[f] = counts

```

```

# Plot all four histograms in one figure

```

```

fig, axes = plt.subplots(1, 4, figsize=(16, 4))

```

```

for ax, f in zip(axes, f_types):

```

```

    plot_histogram(results[f], ax=ax, title=f)

```

```

plt.tight_layout()

```

```

plt.show()

```